

# Design of High Speed Relational Database Based on Labview TDMS Format

Jinwei Lin<sup>1,a</sup><sup>1</sup>Monash University, Bandar Sunway, Subang Jaya, Malaysia<sup>a</sup>e-mail: jlin0119@student.monash.eduJinbin Lin<sup>2,b</sup><sup>2</sup> Shenzhen Bao 'an Water Group, Baoan, Shenzhen, China<sup>b</sup>e-mail: linjinbingo@163.com

**Abstract**— In this paper, a new database design idea based on TDMS format of Labview is proposed that has been successfully programmed and practiced. Firstly, the paper will discuss the respective characteristics and differences between the using of third-party database and the using of file IO operations when the Labview program needs to meet the requirement of the storage of data. Then the paper will discuss how to use TDMS format to design a database based on TDMS, and the characteristics and basic ideas of this kind of native high-speed TDMS database supported by Labview. Next, in the part of the research and implementation methods, the paper will present how to use the make the TDMS database to support the basic operations of CRUD in relationship database. Following, a data processing interpreter whose core is the conversion between the TDMS format data and two-dimensional array is proposed, which has made the more complex operations of TDMS data realized. Subsequently, this paper shows how to add native SQL language support to the TDMS database. Finally, a comprehensive analysis and summary of database design using TDMS data format is made.

**Keywords**- Array; CRUD; TDMS; SQL

## I. INTRODUCTION

Labview is a graphical programming software that released by National Instruments (NI), which has a wide range of industrial applications. In industrial application design cases, data acquisition and processing and storage modules are often used (Zhang et al., 2014). As a graphic programming language, Labview is mainly designed for the industrial and scientific test and measurement design and programming (Shao & Gao et al., 2014), which often have the requirements of retrieve and storage of big data with a high-speed. For instance, the design of module with a high-speed measurement and storage for signal data in communications (Zlatev, 2019), or the real-time recording and the analysis for medical measured data (Swain et al., 2021). To satisfy the requirements of collecting, storage, retrieving and analysis a large amount of data, Labview provides a variety of modules for data storage. There are two main methods to store the data with real files in Labview. One is to use the file IO to save the data. The other is use a third-party databases plug-in to edit and store the data (NI, 2021).

For the general data storage needs, Labview's file read and write IOs modules can be useful (NI, 2021). The four formats of files that can be read and wrote and edited by Labview are: ASCII, Binary, XML, TDMS (NI, 2021). Among the four file formats, TDMS files have the fastest read and write speed for data processing and are often used and promoted for the design that need to process large amounts of data in a high-speed (NI, 2021). However, through the experiments, it is be proved that these four file formats do not have the general database edit

functions of add, delete, change and search. But the general databases can provide the functions of fast data editing, storage, access, query, update and search and other more complex and convenient functions. Therefore, to satisfy the requirements users to store and retrieve big data with a third-party database, Labview provides some plug-ins that can achieve the basic database operations (NI, 2021). However, due to the lack of connection stability and read-write speed limitation between Labview program and database, third-party databases are not first choice in high speed situations. On the other hand, Labview has designed and developed the operation modules of TDMS format files. The TDMS file format is the fastest file format that supports high-speed reading and writing among the four file formats (NI, 2021). Therefore, a reasonable idea is to combine TDMS basic operating functions and the characteristics of relationship database, to design and develop a TDMS database that is natively supported by Labview program rather than a third-party component. This TDMS database should has the characteristics of TDMS, which means to support the high-speed data storage and retrieve, and will be very in line with the actual needs and useful for Labview programming.

The purpose of this paper is to design and develop a TDMS database prototype with the functions of general basic relational database based on the characteristics and principles of TDMS format file of Labview. The final design model should satisfy the CRUD (Create, Retrieve, Update, Delete) functions in the data processing functions of the basic relational databases. At the same time, based on the close operation and connection relationships between the SQL language and relational database infrastructures, this paper will finally put forward a new idea on how to add SQL interpreter to the TDMS database model which has realized the basic CRUD functions, to make the database to support the exploration and experiment of SQL language. All the experimental graphical programs in this paper have been tested in Labview2021 community edition.

## II. MATERIALS AND METHODS

### A. TDMS Features to Database

In order to develop a relational database that has the corresponding CRUD mechanism and functions based on TDMS file format, the first step is to analysis the differences and similarities of data CRUD operations between TDMS file and relational database. Then on this basis to seek the design and optimization methods. In the data format of TDMS: A TDMS file can contain one or more groups, and a group can contain one or more channels (NI, 2021). In the module TDMS file IO plate of Labview, the attribute setting and acquisition of

TDMS files are also accessible. Although Labview can support setting and obtaining the attributes of TDMS files, it does not support the operation of deleting. Moreover, if the attribute values of TDMS files are used as the operated meta-objects, it will be difficult to reflect the advantages of high-speed channel reading and writing that are brought by Labview. Therefore, other conversion mechanisms are needed. Through experiments, the following conversion mechanisms is suitable: the data type of a group in the TDMS file format can correspond to a row in a basic relational database. A channel in the TDMS corresponds to a column in a basic relational database. A TDMS file corresponds to a table in a basic relational database.

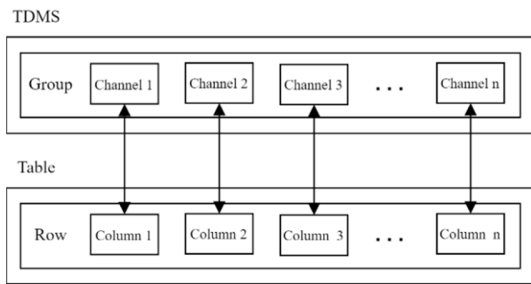


Figure.1 Correspondence between TDMS and table

As shown in Fig. 1, in study of this paper, a TDMS file is considered to be a table in a database. In a relational database, a table is used as a base operation object unit, and in this TDMS database, a TDMS file is used as a base operation object unit.

### B. Open and Create

To design a relational database, the first thing to design is to open and create a table. As shown in Fig. 2, this module is designed based on the open module of TDMS.

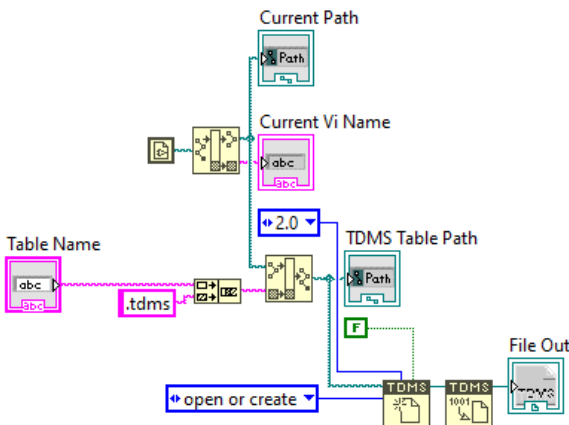


Figure.2 TDMS database open or create table

This module is named VI\_OOC for illustration purposes. To use this module, just need to simply enter the name of the table the wanted to open or create. The Create or Replace module can be obtained by setting the operation option in the TDMS open module in the above module to make the functions of Create or Replace. And the Create or Replace module was named as VI\_COR, which mainly be used to replace table files.

### C. Delete and Clear

The functions of deletion or clearing are also an important in database design. As shown in Fig. 3, the design and implementation of table files deletion function of this TDMS database is directly use the general deletion control of advanced file IO in Labview. Deletion after the close of TDMS files.

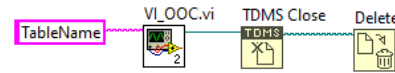


Figure.3 TDMS database delete table

Another way to clear a TDMS file is to use the TDMS deletion control, which can empty all or some of the rows and columns of the corresponding database table file. Therefore, based on the TDMS delete control, it is able to define a sub-VI that removes all rows and columns in the table through a loop.

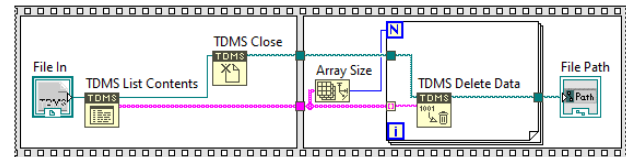


Figure.4 TDMS database clear table

The clear module is named VI\_ClearT. The Flat Sequence Structure is added to make TDMS file close first and then be cleared. File In and File Out represent input and output handles of TDMS file processing, respectively. Note that only the data of channels can be deleted by using this method. The original group and channel names of TDMS files cannot be deleted, but the new data can be re-read and re-written as well as updates to the original data, which is called overwriting.

### D. Create Row Sequentially

After completing the study of operations on TDMS table unit, the next step is to research the operations on rows and columns of the table. In this TDMS database, a row is a group, and a column is a channel. Combined with the TDMS open control in the TDMS panel, the TDMS single-line write sub-VI as shown in Fig. 5 is developed.

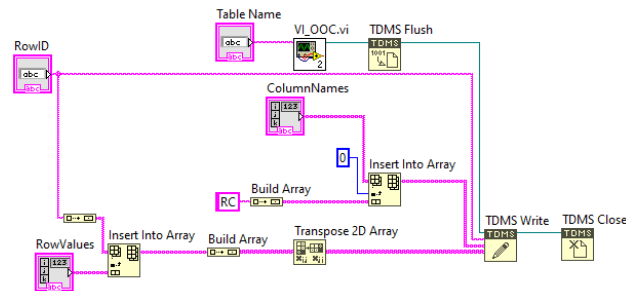


Figure.5 TDMS database creating one row

The above program implements the sequential operation of writing a row of data to a specific table in the TDMS database. Calling the above program multiple times in a particular table can write the data row by row in the table. The value of row 0,

column 0 in the table is the string constant RC, which is used as a flag and can be changed. After implementing row-by-row writing in the table of the TDMS database, the next step will be research how to insert the specific i-th row into the specific index. After studying TDMS operation controls, it is found that only the TDMS Advanced Asynchronous read-write module in TDMS panel can set the position of the next reading or writing. However, it supports only 1D or 2D array of integers, floating-point numbers, Booleans, and timestamps, and does not support string data type. This problem can be realized by the type conversion between floating point data and string data, which is converting string to binary and then converting binary to general float type numbers. However, this realization idea is not discussed in this paper due to the impact on low efficiency.

### E. Table and Array

To insert rows at a specific location in the TDMS database, or to perform more complex database operations, this paper designed a transformation processing mechanism with array processing. This processing mechanism can be called array conversion interpreter processing for TDMS databases.

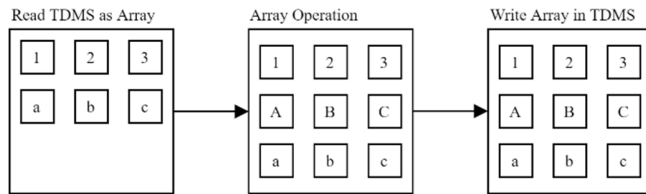


Figure.6 TDMS to array handling mechanism

As shown in Fig. 6, when performing complex table operations, the TDMS database will first read the DATA from the TDMS file, then convert the TDMS data into a two-dimensional array, and perform operations on the data, such as inserting a specific row of data into a specific row. The manipulated array data is then written to the previous TDMS file. Therefore, this conversion processing mechanism requires two core processing modules, one to convert the TDMS file data into an array and the other to write the processed array data to the previous TDMS file.

### F. Read TDMS as Array

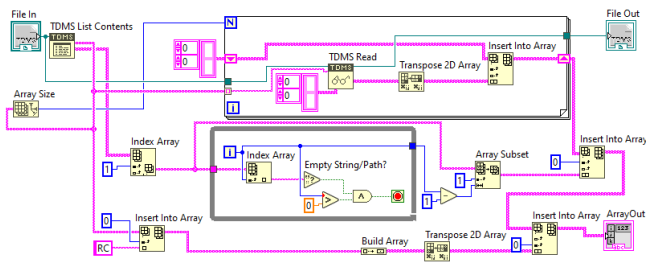


Figure.7 Reading TDMS as array

As the TDMS control provided by Labview is used to write TDMS file data, the group name of data is indispensable. For precise read and write operations, both the group name and channel name must be provided. As shown in Fig. 7, the group name and channel name are first obtained through the TDMS

List Contents control.

### G. Write Array in TDMS

The data read from the TDMS file will be written back to the previous TDMS file after being converted to an array and processed. Before writing the previous TDMS file, the previous TDMS need to be cleared by connecting the developed sub-VI named VI\_ClearT. Then writing the new data after processing. Fig. 8 shows the design of VI\_ClearT that handles the writing the processed array data to the previous TDMS file.

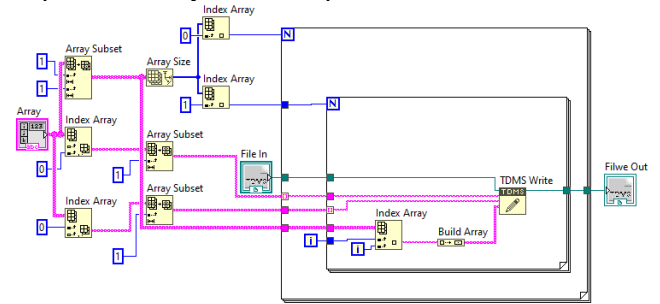


Figure.8 Write array in TDMS

### H. Basic Database Array Processing

After the completing the design of the interpreter for reading and writing conversions between TDMS and arrays. The common design of handling the rows and columns processing of the TDMS database table corresponding to the intermediate array will be shown as follow.

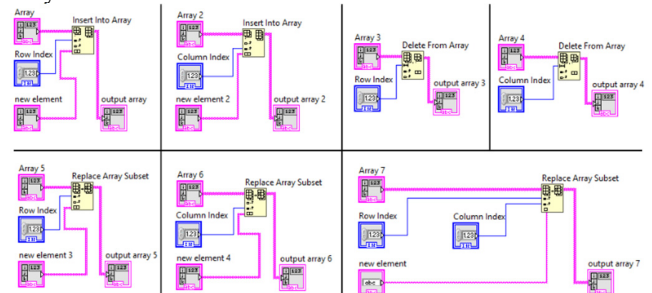


Figure.9 Write array in TDMS

The subgraph in Fig. 9 shows the operations of inserting, deleting, and updating on rows and columns and cells of tables in the TDMS database, respectively. As shown in Fig. 10, the row by row search and column by column search, and the traversal of indexes for rows and columns of a particular Cell are performed on a table in the TDMS database.

By the interpreter between the array data and TDMS data, more complex operations or implementations of database can be achieved which will get rid of inherent limitations of TDMS original operation controls. This processing happens in the middle procedure between read and write TDMS data.

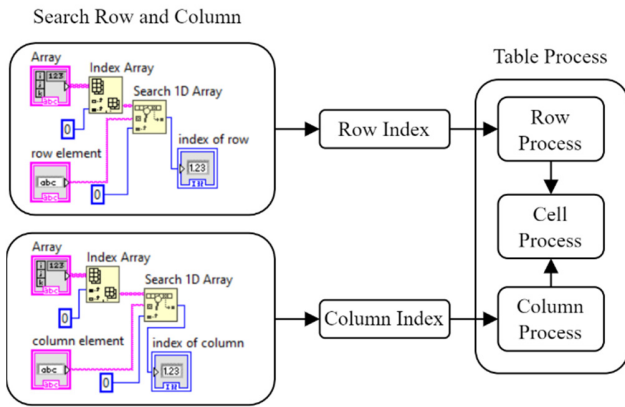


Figure.10 Searching and processing

### I. Basic Database Array Processing

After completing the basic design of TDMS database, this session of the paper will introduce the extension experiment of SQL language support, and gets the design idea of how to expand SQL support for the current TDMS database. SQL is an important component of database design (Bouamrane et al., 2021). As shown in Fig. 11, this TDMS database supports the design of select-from statements in SQL syntax. The processing module is to support the SQL code such as: SELECT Columns FROM Table. The core module of the whole interpreter is Match Pattern. After the SQL code to be analyzed is introduced into the SQL String end. If a SELECT reserved word is detected in the code, the subsequent analysis part will be continued to find the names of columns and the name of table.

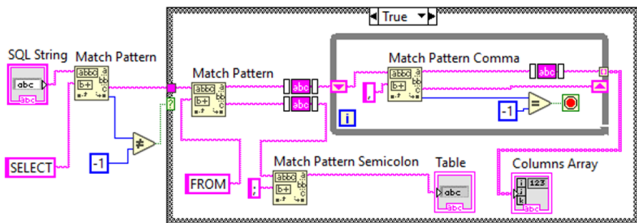


Figure.11 SQL Interpreter for SELECT FROM

The above interpreter will output the array of the names of columns and the name of table in the SQL code. If there is only one column, the output column array has only one element. The comma and semicolon will be important syntax symbols. When the SQL interpreter find the terminal command context, the TDMS database will call the corresponding processing.

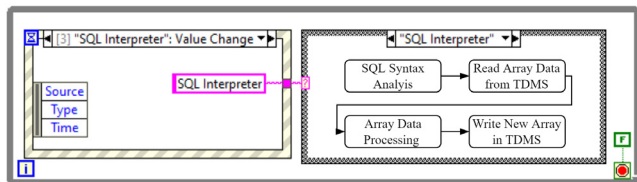


Figure.12 SQL command case processing

As shown in Fig. 12, the basic processing of the SQL interpreter is as follows: firstly, the main code will call the SQL interpreter module, then the SQL interpreter will analyze the

Syntax of the SQL code, followed by the reading and processing and re-writing of the TDMS data with the process type of array.

## III. RESULTS AND DISCUSSION

All the graphic programs of this paper are proved effective and can be directly imported as reference in experiments. The next part of this study will be a design and research on how to combine the all parts in to a specific project. The advantage of this research is using the special characteristics of TDMS to develop a native supporting database for Labview program. The conversed processing mechanism between array data and TDMS data is proved useful and effective. The basic CRUD operations of the TDMS database are successfully implemented. In the future and the further level developments, the network connection function part should be considered as an expanded exploration, which will make the TDMS database more useful.

To test the retrieving and writing performance of this TDMS database, the following experiments as shown in Fig. 13 are implemented. In the channel box, inputting the specific number of channels, followed by running the program. After executing the program, the time that the TDMS database used to writing and retrieving the data is provided. The size of the input array is: 3×channels.

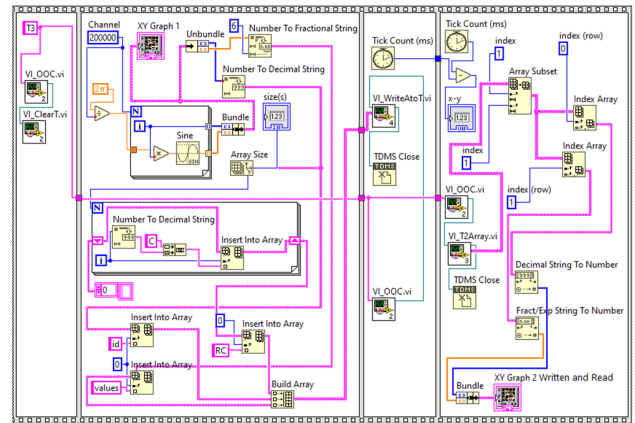


Figure.13 Writing and Reading Experiments

The results of this experiment are influenced and restricted by the computing power and memory size of the executing computer. As shown in Fig. 14, the capacity of one million channels is available in the condition of 32G RAM computing.

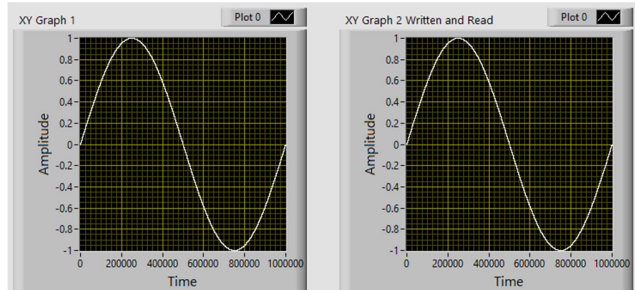


Figure.14 Comparison between original and database retrieving

To better test the retrieving and writing performance of the TDMS database. Eight testing experiments with the program as



shown in Fig. 13 are implemented. For each experiment, the used-exhausted time and the mean value in milliseconds is shown. The data size is defined by the numbers of channels. The serial performance results are shown in Table 1.

TABLE 1 RUNNING TIME

| Data Size<br>/channel | Run Time for Writing and Reading Once /millisecond |       |       |       |       |       |       |       |           |
|-----------------------|----------------------------------------------------|-------|-------|-------|-------|-------|-------|-------|-----------|
|                       | time1                                              | time2 | time3 | time4 | time5 | time6 | time7 | time8 | time mean |
| 100                   | 49                                                 | 48    | 49    | 41    | 40    | 49    | 40    | 56    | 46.5      |
| 200                   | 48                                                 | 64    | 56    | 64    | 56    | 57    | 56    | 55    | 57.0      |
| 500                   | 81                                                 | 78    | 69    | 81    | 77    | 78    | 77    | 79    | 77.5      |
| 1000                  | 95                                                 | 114   | 105   | 98    | 114   | 107   | 106   | 114   | 106.6     |
| 2000                  | 162                                                | 178   | 162   | 178   | 169   | 165   | 173   | 198   | 173.1     |
| 5000                  | 312                                                | 361   | 320   | 344   | 379   | 338   | 385   | 344   | 347.9     |
| 10000                 | 662                                                | 639   | 714   | 649   | 693   | 696   | 715   | 660   | 678.5     |
| 20000                 | 1189                                               | 1273  | 1257  | 1240  | 1244  | 1252  | 1286  | 1278  | 1252.4    |
| 50000                 | 2754                                               | 2945  | 3001  | 3079  | 2926  | 3154  | 3071  | 2920  | 2981.3    |
| 100000                | 5471                                               | 5910  | 5994  | 6053  | 5931  | 6066  | 6060  | 6019  | 5938.0    |
| 200000                | 10589                                              | 13826 | 12052 | 12517 | 11988 | 12589 | 11976 | 12751 | 12286.0   |
| 500000                | 26371                                              | 30607 | 30518 | 30624 | 30276 | 30004 | 30105 | 29549 | 29756.8   |

Subsequently, for better understanding and analyzing the relationship between the reading and writing rates of the TDMS database and the number of the channels used that is related to the data size, the X-Y relationship figure is shown as Fig. 15.

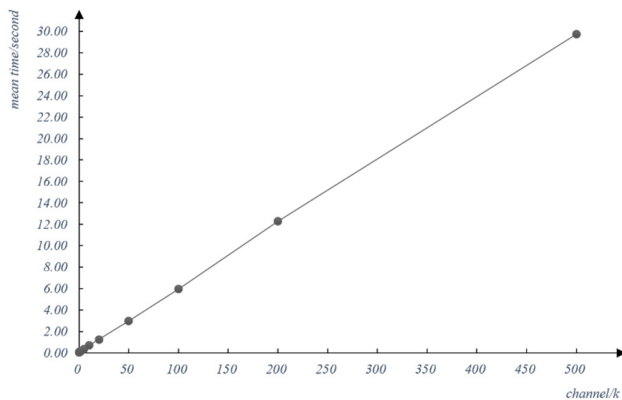


Figure.15 TDMS database reading and writing rates

As shown in Fig. 15, there is a near linear relation that is increasing between the reading and writing rates of the TDMS database and the number of the channels, which means the operating rate of retrieving and writing of the TDMS database is stable. The rate is one second for 20 thousand channels in both implementing the retrieving and writing of the data. Therefore, as the official default fastest retrieving and writing file format, the researches and explorations in TDMS database is reasonable and prospective, compared with the most normal database that need to spend time in third-party connections. Furthermore, the TDMS database will be easier supported by Labview programs or related implementations, which is another advantaged for using the TDMS database.

#### IV. CONCLUSION

This paper has proved that the combination of TDMS and array processing can be used to develop a relationship database

by a series of implementations and experiment programs. This kind of design of database is supported natively by Labview program, moreover, the TDMS database can be expanded as a third-party database and called by other kinds of program with the SQL code interpreter developed. There are not too much researches on the native Labview database development and study, so this TDMS database that include the characteristic of the fastest files processing speed of Labview will provide a new design idea for Labview local database. Based on this research, by the deeper researches of the development and study for the SQL interpreter module, the functions of this database will become more and more comprehensive and robust.

#### REFERENCES

- [1] Zhang, Jian Feng, Ren, Gao Feng, Qian, Zhao Ming, & Zhang, Sa Sa. (2014) Information Monitoring and Management System for Tailings Based on LabVIEW. *Applied Mechanics and Materials*, 484-485, 400-403.
- [2] Shao, Jian Long, & Gao, Jun Fang. (2014) A Design of Data Fusion Management System Based on LabVIEW. *Advanced Materials Research*, 945-949, 3356-3359.
- [3] Zlatev, Zoran, & Hinov, Nikolay. (2019) Signal Processing and Analysing Big Mass Data Using LabView. *TEM Journal*, 8(2), 617-622.
- [4] Swain, Kunjabihari, Cherukuri, Murthy, Mishra, Sunil Kumar, Appasani, Bhargav, Patnaik, Suprava, & Bizon, Nicu. (2021) Li-care: A labview and iot based ehealth monitoring system. *Electronics (Basel)*, 10(24), 3137.
- [5] Bouamrane, Karim, Matallah, Houcine, & Belalem, Ghalem. (2021) Comparative Study Between the MySQL Relational Database and the MongoDB NoSQL Database. *International Journal of Software Science and Computational Intelligence*, 13(3), 38-63.
- [6] National Instruments Corporation. (2021) LabVIEW Support, The NI TDMS File Format, <https://www.ni.com/en-us/support/documentation/supplemental/06/the-ni-tdms-file-format.html>